Computer Engineering Department

Software Engineering (BETCE15322)

Submitted By

Divyaraj Chavda

(240905040018)

2026-27

Gyanmanjari Innovative University

Gyanmanjari Innovative University



Certificate

This is to certify that Mr. **Chavda Divyaraj S**. of B,tech. ***5th*** semester, **Computer Engineering**, Enrolment No. **240905040018** has satisfactorily completed his practical work of subject **Software Engineering-(BETCE15322)** during the academic year **2026-27** submitted on _____.

Date: _____

Faculty In charge **Head of department**

Subject :Software Engineering **Year : 2026-27**

Enrollment No.: 240905040018 **Name: Chavda Divyaraj S.**

Index

Sr. No.	Practical Title	Page No.	Date of Performance	Date of Submission	Sign
1	Study the complete Software Development Life Cycle (SDLC) analyze various activities conducted as a part of various phases.				
2	Develop requirements specification for a given problem.				
3	Develop UML Use case diagram for a problem.				
4	Develop UML Activity and Swim Lane diagram for a problem.				
5	Develop UML Class diagram for a problem.				
6	Develop UML Sequence diagram for a problem.				
7	Develop UML Data Flow diagram for a problem.				
8	Develop UML ER diagram for a problem.				
9	Estimation of Test Coverage and structural complexity.				
10	Design and develop test suites for a given software module by applying various testing techniques.				
11	Test the login, Registration (Sign Up), Sign-in functionality of an OAuth module.				
12	Use the Blisk tool to test the layout and responsiveness of your website or personal project.				
13	Generate a bug report of any given definition.				

Practical – 1

Aim: - Study the complete Software Development Life Cycle (SDLC) analyze various activities conducted as a part of various phases.

❖ Introduction: -

The **Software Development Life Cycle (SDLC)** is a step by step process which is used to plan, build, test, deliver and maintain software. The work is divided into phases, and every phase has its own activities and its own output. In this journal the SDLC is studied with one project, **LearnSphere**, an Online Course / E-Learning Platform where students can enroll in courses, watch lectures, give quizzes and get certificates. The same project is used in all the practicals.

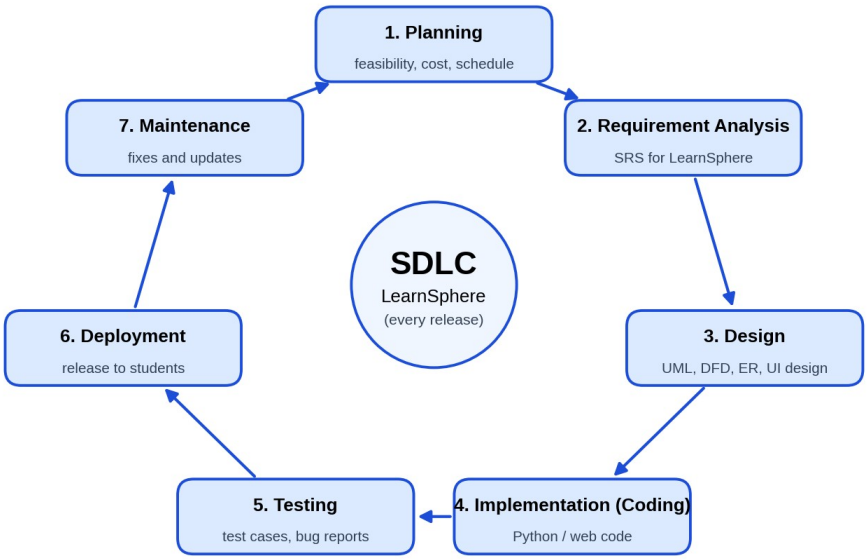


Fig 1: Phases of SDLC for LearnSphere

❖ Phases of SDLC and activities conducted in each phase: -

Phase	Activities conducted in LearnSphere	Output (Deliverable)
1. Planning	Feasibility study (technical, economic, operational), deciding the main features, estimating cost and time, and finding risks like payment failure and video storage.	Project plan, feasibility report
2. Requirement Analysis	Talking to students and instructors, collecting what they need (course search, video lectures, quiz, certificate, online payment), then writing and checking the requirements.	Software Requirement Specification (SRS)
3. Design	Designing the architecture, database (ER diagram), data flow (DFD), UML diagrams and the screens like home page, course page and student dashboard.	Design document (HLD and LLD), UI designs

Phase	Activities conducted in LearnSphere	Output (Deliverable)
4. Implementation (Coding)	Writing the code of registration, login, course catalogue, payment and quiz modules, following coding standards, code review and version control (Git).	Source code
5. Testing	Unit testing of every module, integration testing of enroll and payment, system testing, testing on different devices and browsers, logging and fixing bugs.	Test cases, test report, bug reports
6. Deployment	Hosting the website on a server, setting up the database and the payment gateway, and giving training to instructors and admin.	Live system, user manual
7. Maintenance	Fixing the bugs reported by users, adding new features like discussion forum, updating courses and taking regular backups.	Patches, updates, new versions

❖ SDLC Models: -

Model	Description	Suitable when
Waterfall	Linear and sequential. A phase is started only after the previous phase is completed.	Requirements are clear and fixed; small projects
Iterative / Incremental	Software is built in small increments and each increment adds more functionality.	A working version is needed early
Spiral	Combines iterations with risk analysis done in every loop of the spiral.	Large and high-risk projects
V-Model	Every development phase has a matching testing phase (verification and validation).	High reliability is needed
Agile	Short development cycles (sprints) with continuous feedback from the customer.	Requirements change frequently

Model selected for LearnSphere: Agile (Scrum). Features like quiz, certificate and discussion forum can be added in small sprints, and the feedback of students can be used in the next sprint.

❖ Benefits of SDLC: -

- Gives a clear structure and a clear plan to the whole project.
- Every phase has a defined output, so progress can be measured and controlled.
- Errors are found early, which reduces the cost of fixing them.
- Improves communication between the customer, developers and testers.
- Gives a high-quality product that is delivered on time and within budget.

❖ Conclusion: -

The complete SDLC was studied with the LearnSphere project. The seven phases (Planning, Requirement Analysis, Design, Implementation, Testing, Deployment and Maintenance), their activities and their outputs were analysed.

Practical – 2

Aim: - Develop requirements specification for a given problem.

❖ Problem Statement: -

LearnSphere – Online Course / E-Learning Platform. Students can search and enroll in courses, pay online, watch video lectures, attempt quizzes and download a certificate. Instructors create courses and upload lectures. The Admin manages users and approves courses.

❖ Software Requirement Specification (SRS): -

❖ 1. Introduction

- **Purpose:** To describe the requirements of the LearnSphere platform for the developers, testers and the college / client.
- **Scope:** The system allows students to learn online, instructors to publish courses, and the admin to control the whole platform.
- **Intended audience:** Developers, testers, project manager, instructors and students.
- **Definitions:** SRS = Software Requirement Specification, FR / NFR = Functional / Non-Functional Requirement, OTP = One Time Password used to verify the e-mail.

❖ 2. Overall Description

- **Product perspective:** A new, self-contained web and mobile application that works with a payment gateway, Google sign-in and an e-mail service.
- **Product functions:** Registration and login, course search, enrollment with payment, video lectures, quiz and assignments, progress tracking, certificate, course management and reports.
- **Operating environment:** Web browser (Chrome, Edge, Firefox) and Android/iOS phone; server with a relational database and video storage.
- **Constraints:** Internet connection is needed, and payments must follow the rules of the payment gateway.
- **Assumptions:** Students have a smartphone or computer, and instructors upload videos in a supported format (MP4).

User class	Description
Student	Registers, searches and enrolls in courses, pays, watches lectures, gives quizzes and downloads certificate.
Instructor	Creates courses, uploads lectures and quizzes, and evaluates assignments.
Admin	Approves courses, manages users and generates reports.
Payment Gateway	External system that processes the online payment.

❖ 3. Specific Requirements

❖ 3.1 Functional Requirements

ID	Requirement	Priority
FR-1	The system shall allow a new user to register using name, e-mail and password, and verify the e-mail with an OTP.	High
FR-2	The system shall allow a registered user to log in and log out.	High
FR-3	The system shall allow login with a Google account (OAuth 2.0).	Medium

ID	Requirement	Priority
FR-4	The system shall allow the student to search courses by name, category or price.	High
FR-5	The system shall allow the student to enroll in a course after online payment (UPI, card, net banking).	High
FR-6	The system shall allow the student to apply a coupon code (maximum 50% discount).	Medium
FR-7	The system shall allow the student to watch lectures and continue from the last watched position.	High
FR-8	The system shall allow the student to attempt quizzes and see the score immediately.	High
FR-9	The system shall allow the student to submit assignments and the instructor to evaluate them.	Medium
FR-10	The system shall show the progress of every enrolled course.	Medium
FR-11	The system shall issue a certificate when progress is 80% or more, quiz score is 50 or more and all assignments are submitted.	High
FR-12	The system shall allow the instructor to create, update and delete courses and to upload lectures.	High
FR-13	The system shall allow the admin to approve courses, manage users and generate reports.	Medium

❖ 3.2 External Interface Requirements

- **User interface:** Responsive pages with a search bar, course cards, video player and student dashboard.
- **Hardware interface:** Smartphone or computer for users, and a server with enough storage for videos.
- **Software interface:** Payment gateway API, Google OAuth, e-mail service and a video streaming service.
- **Communication interface:** HTTPS protocol is used for all communication.

❖ 3.3 Non-Functional Requirements

Type	Requirement
Performance	A page shall load within 3 seconds, and the system shall support 2000 users at the same time.
Security	Passwords shall be stored in hashed form, OTP shall expire in 5 minutes, an account shall be locked for 15 minutes after 5 wrong passwords, and a login session shall expire after 30 minutes.
Reliability	The system shall not lose enrollment or payment data, even after a failure.
Usability	A new student shall be able to find and enroll in a course in less than 5 minutes without training.
Portability	The system shall work on all major browsers and on Android and iOS.
Responsiveness	The website shall work properly on screens from 360 px width (mobile) to desktop.

❖ **Conclusion: -**

The SRS of the LearnSphere platform was developed with the introduction, overall description, and functional, interface and non-functional requirements.

Practical – 3

Aim: - Develop UML Use case diagram for a problem.

❖ Problem: -

LearnSphere. A Student can register or log in, search courses, enroll in a course (this includes making the payment), watch lectures, attempt quizzes and download a certificate. An Instructor creates courses, uploads lectures and evaluates assignments. The Admin manages users, approves courses and views reports. The Payment Gateway is an outside system that processes the payment.

Actor	Use cases
Student	Register / Login, Browse and Search Courses, Watch Lectures, Enroll in Course, Attempt Quiz, Download Certificate
Instructor	Create Course, Upload Lectures, Evaluate Assignments
Admin	Manage Users, Approve Courses, View Reports
Payment Gateway	Make Payment (included in Enroll in Course)

❖ Use Case Diagram: -

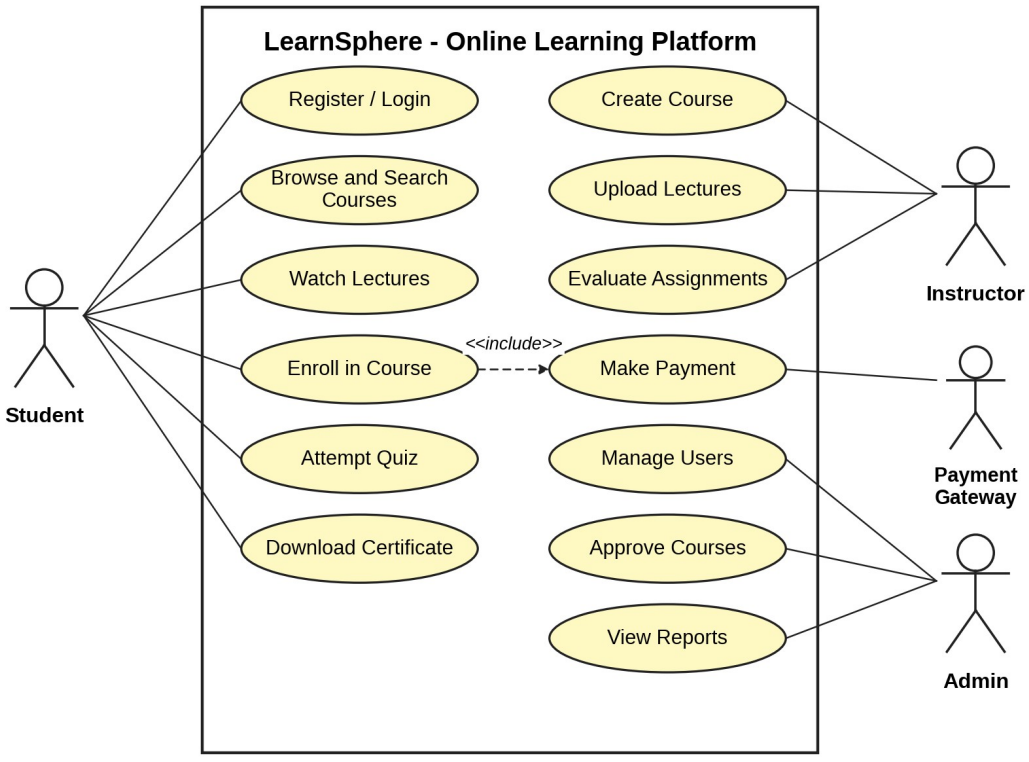


Fig 1: Use Case Diagram of LearnSphere

Practical – 4

Aim: - Develop UML Activity and Swim Lane diagram for a problem.

❖ Problem: -

Course Enrollment and Payment. The student selects a course and clicks Enroll. If the student is not logged in, login or registration is done first. Then the student chooses a payment method and pays through the payment gateway. If the payment is successful, the enrollment is saved, a confirmation mail is sent and the student can open the course. If the payment fails, the student can try again.

❖ Activity Diagram: -

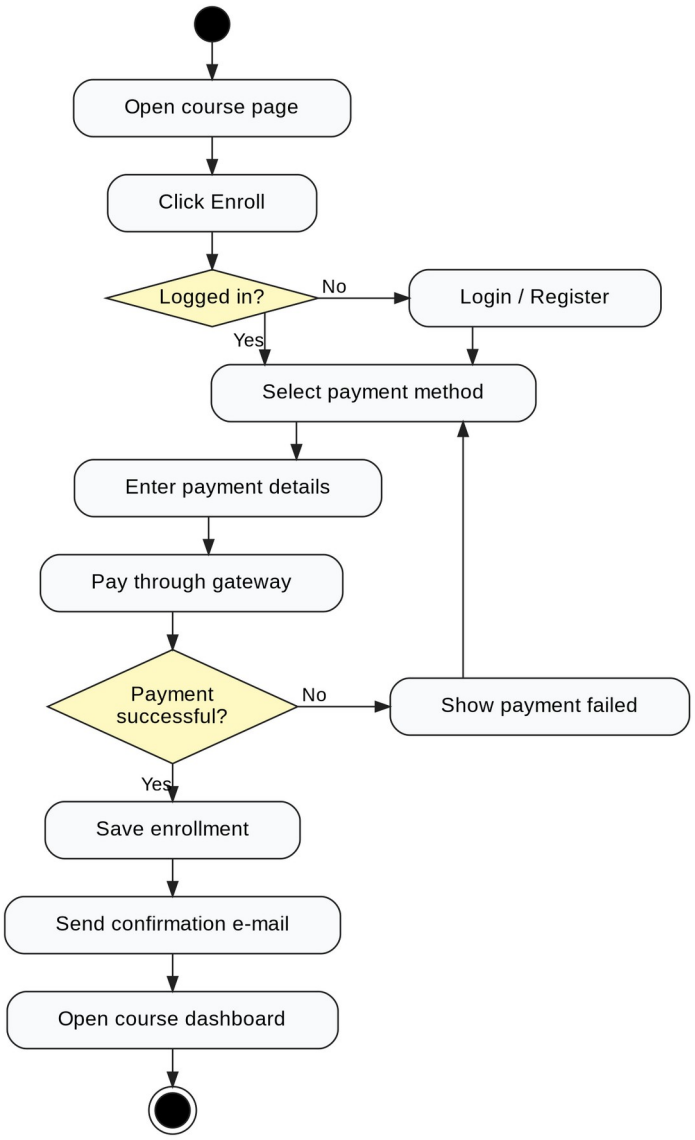


Fig 1: Activity Diagram of Course Enrollment and Payment

❖ Swim Lane Diagram: -

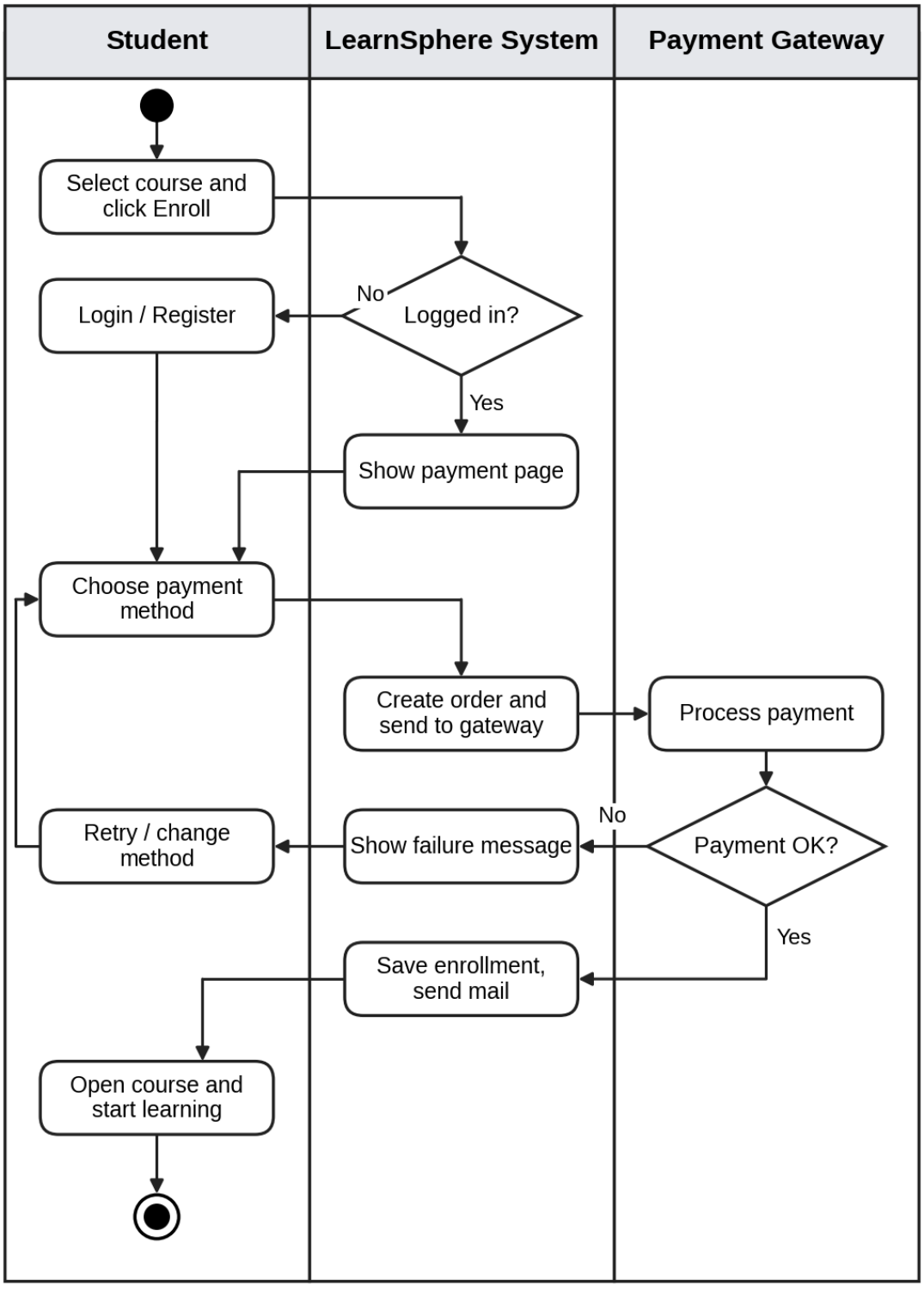


Fig 2: Swim Lane Diagram of Course Enrollment and Payment

Practical – 5

Aim: - Develop UML Class diagram for a problem.

❖ Problem: -

LearnSphere. User is the parent class of Student and Instructor. An Instructor creates courses and every Course contains lectures. A Student makes enrollments in courses, every Enrollment is paid through one Payment, and an Enrollment can earn one Certificate.

❖ Class Diagram: -

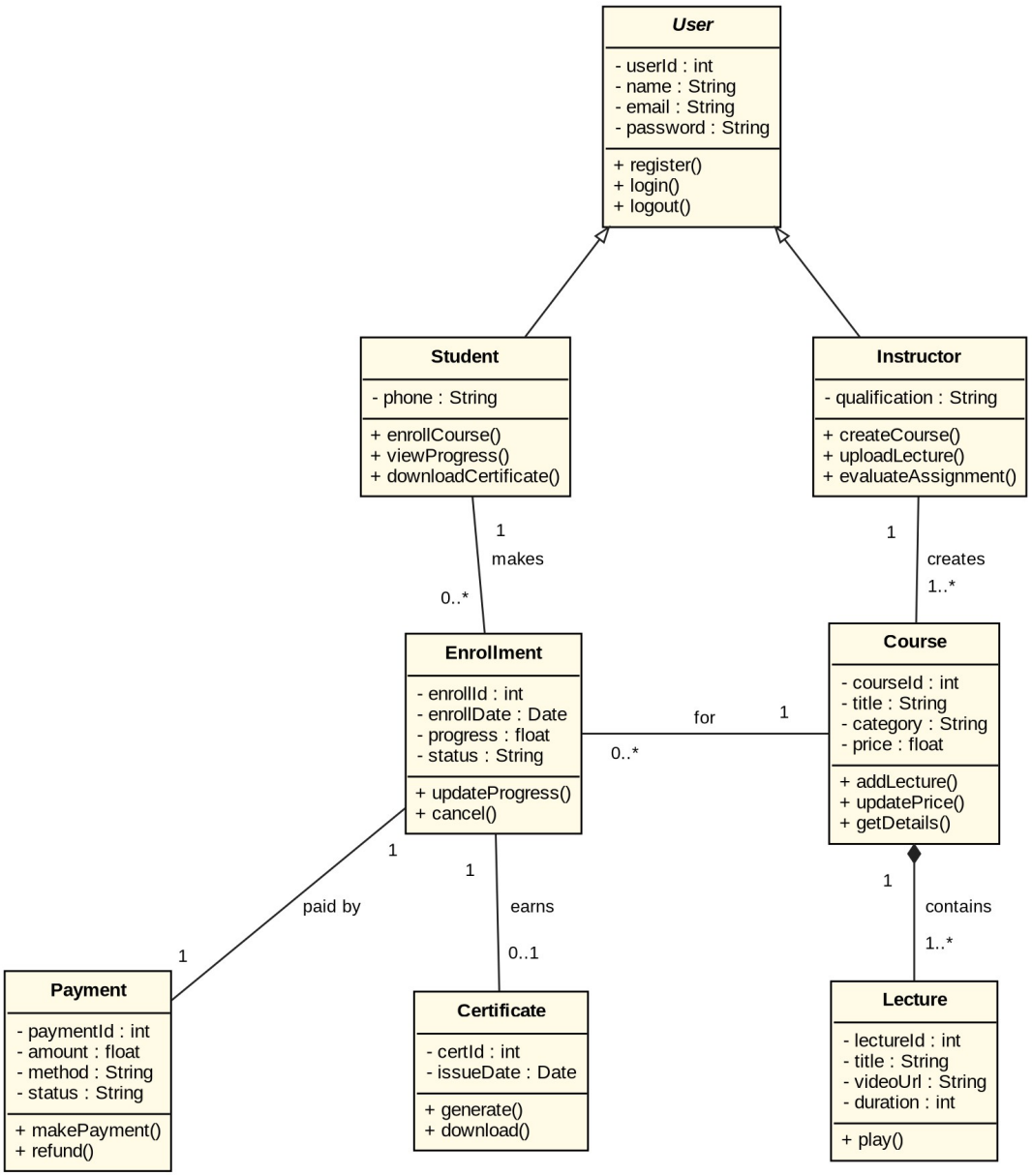


Fig 1: Class Diagram of LearnSphere

Practical – 6

Aim: - Develop UML Sequence diagram for a problem.

❖ Problem: -

Course Enrollment. The student selects a course and the web application gets the course details from the database. The student enrolls, the payment is done through the payment gateway, then the enrollment is saved in the database, a confirmation mail is sent and the student gets access to the course.

❖ Sequence Diagram: -

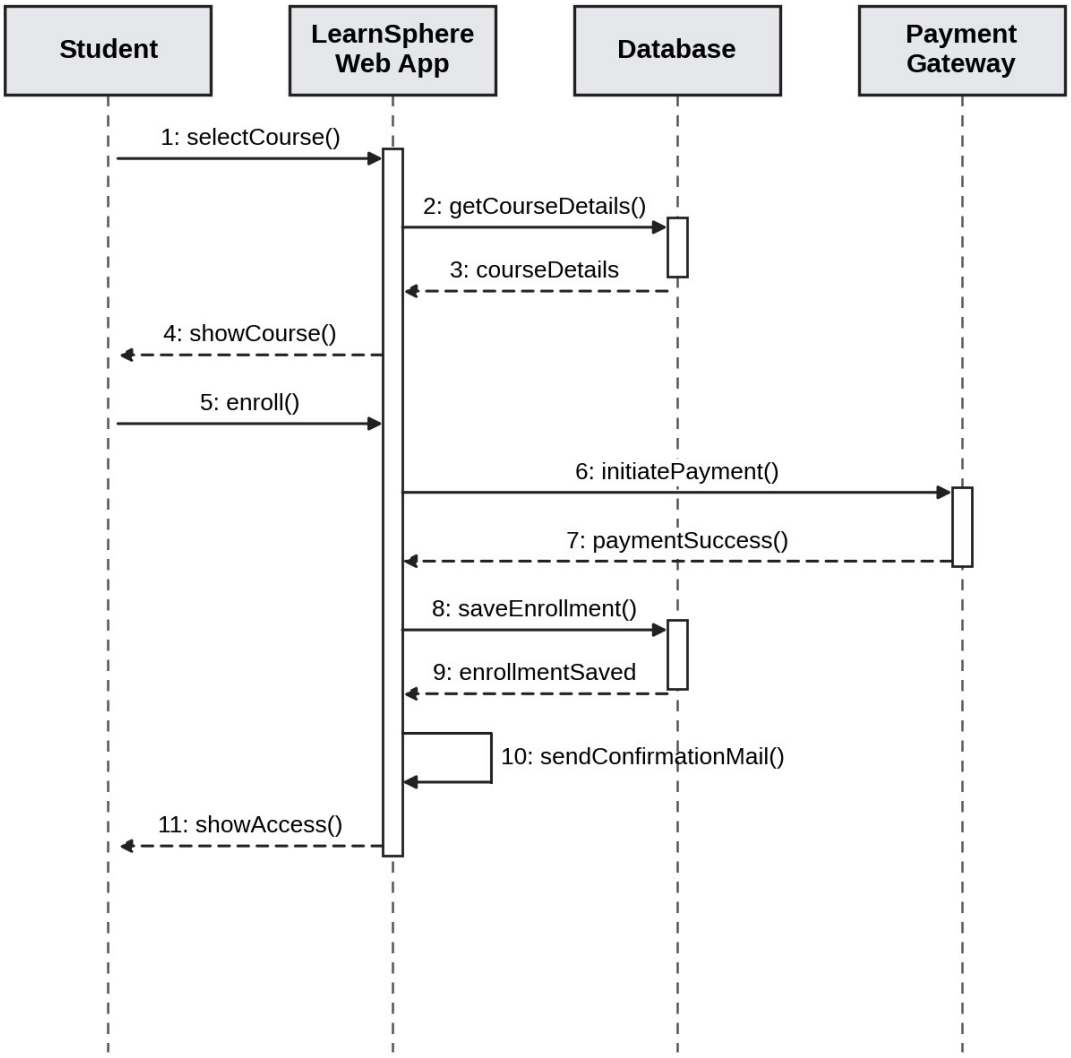


Fig 1: Sequence Diagram of Course Enrollment

Practical – 7

Aim: - Develop UML Data Flow diagram for a problem.

❖ Problem: -

LearnSphere. Students give login and enroll requests and get course content and certificates. Instructors give course and lecture details and get the progress of students. The Admin gives approvals and gets reports. The payment gateway processes the payments. The data is stored in the Users, Courses, Enrollments and Progress stores.

❖ DFD Level 0 (Context Diagram): -

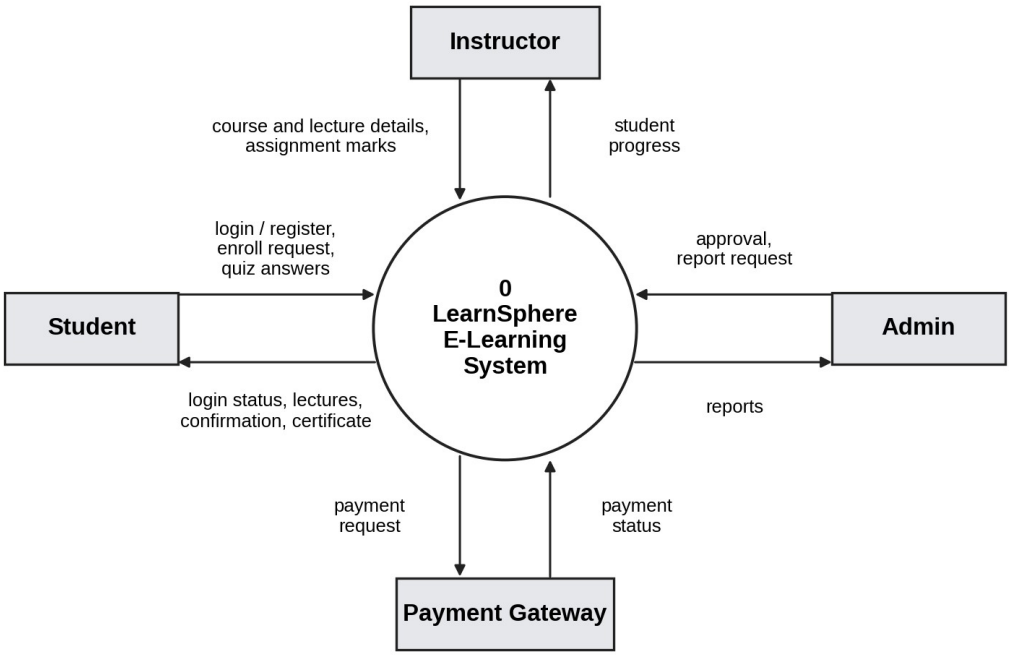


Fig 1: DFD Level 0 of LearnSphere

External entity	Data given to the system	Data received from the system
Student	login / register, enroll request, quiz answers	login status, lectures, confirmation, certificate
Instructor	course and lecture details, assignment marks	student progress
Admin	approval, report request	reports
Payment Gateway	payment status	payment request

❖ DFD Level 1: -

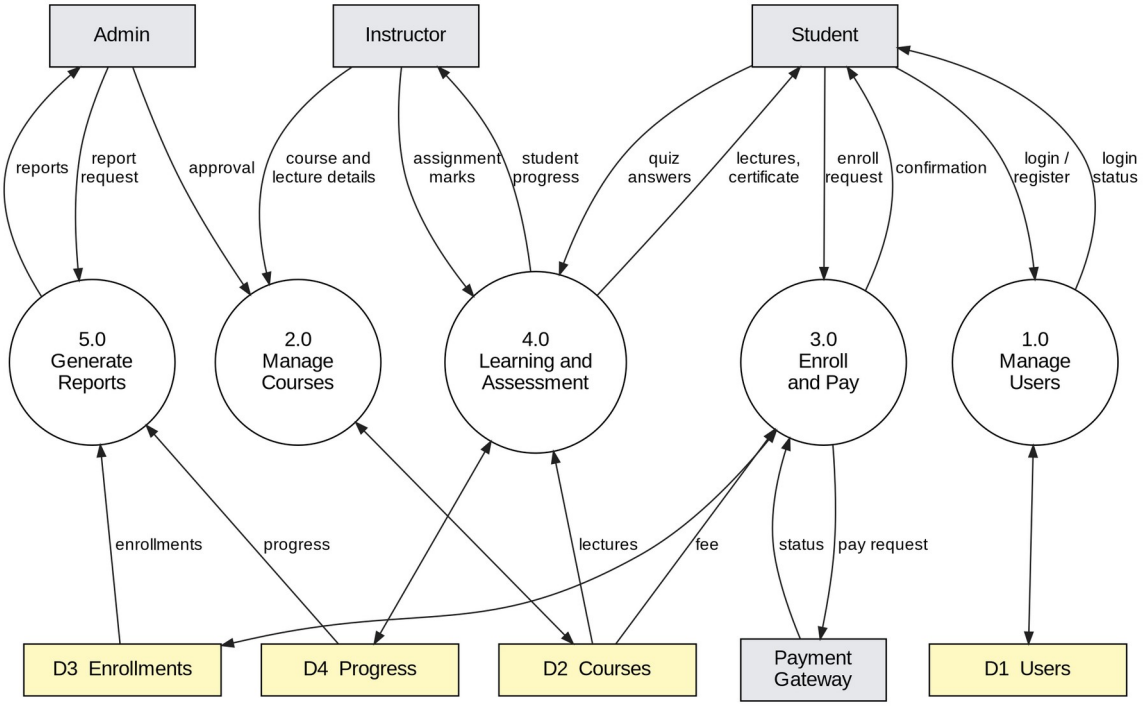


Fig 2: DFD Level 1 of LearnSphere

Process	Description
1.0 Manage Users	Registration, login and OTP verification of users. Uses the store D1 Users.
2.0 Manage Courses	Instructor gives course and lecture details and the admin approves them. Uses D2 Courses.
3.0 Enroll and Pay	Takes the enroll request, sends the payment request to the gateway and saves the enrollment in D3 Enrollments.
4.0 Learning and Assessment	Gives lectures, takes quiz answers and assignment marks, updates D4 Progress and gives the certificate.
5.0 Generate Reports	Reads D3 Enrollments and D4 Progress and gives reports to the admin.

Data store	Contents
D1 Users	user id, name, e-mail, password (hashed), verified status
D2 Courses	course details, price, category, lectures and quizzes
D3 Enrollments	student, course, payment details and enrollment date
D4 Progress	progress percentage, quiz scores and assignment status of every student

The level 1 diagram is balanced with the level 0 diagram: every input and output of the context diagram is also present in the level 1 diagram.

Practical – 8

Aim: - Develop UML ER diagram for a problem.

❖ Problem: -

LearnSphere. One instructor teaches many courses, and one course has many lectures. Many students can enroll in many courses. A student makes many payments, and every payment is made for one course. The system stores the records of instructors, courses, lectures, students and payments. The primary keys are underlined in the diagram.

❖ ER Diagram: -

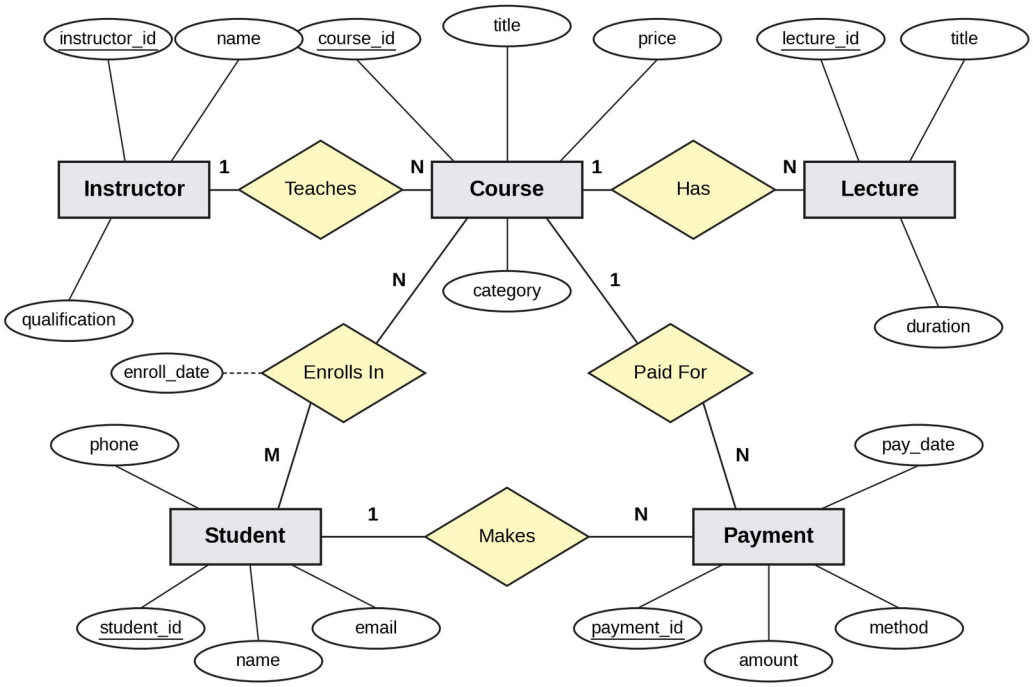


Fig 1: ER Diagram of LearnSphere

Relationship	Entities	Cardinality
Teaches	Instructor – Course	1 : N
Has	Course – Lecture	1 : N
Enrolls In	Student – Course (attribute enroll_date)	M : N
Makes	Student – Payment	1 : N
Paid For	Payment – Course	N : 1

Practical – 9

Aim: - Estimation of Test Coverage and structural complexity.

❖ Problem: -

Find the **cyclomatic complexity** of certificate_status() of LearnSphere and the **statement, branch and path coverage** of its test cases. A certificate is given only if progress is 80% or more, quiz score is 50 or more and assignments are submitted. V(G) = E − N + 2 = P + 1.

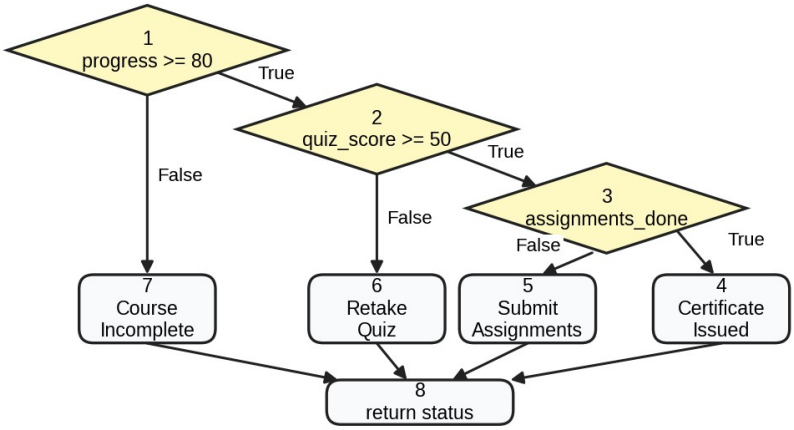


Fig 1: Control Flow Graph of certificate_status() (V(G) = 4)

E = 10, N = 8, so V(G) = 10 − 8 + 2 = 4 (also P + 1 = 3 + 1 = 4). So 4 independent paths and at least 4 test cases are needed.

Path	Nodes	Test case
Path 1	1 → 2 → 3 → 4 → 8	TC1 (90, 70, True)
Path 2	1 → 2 → 3 → 5 → 8	TC2 (90, 70, False)
Path 3	1 → 2 → 6 → 8	TC3 (90, 30, True)
Path 4	1 → 7 → 8	TC4 (50, 70, True)

❖ Program (Python): -

```
"""Practical 9: cyclomatic complexity and test coverage of certificate_status()."""
import ast, inspect, sys

def certificate_status(progress, quiz_score, assignments_done):
    if progress >= 80:
        if quiz_score >= 50:
            if assignments_done:
                status = "Certificate Issued"
            else:
                status = "Submit Assignments"
        else:
            status = "Retake Quiz"
    else:
        status = "Course Incomplete"
    return status

TESTS = [  # name, (progress, quiz score, assignments done), expected result
    ("TC1", (90, 70, True), "Certificate Issued"),
    ("TC2", (90, 70, False), "Submit Assignments"),
    ("TC3", (90, 30, True), "Retake Quiz"),
    ("TC4", (50, 70, True), "Course Incomplete"),
]
```

```
func = ast.parse(inspect.getsource(certificate_status)).body[0]
base = certificate_status.__code__.co_firstlineno - 1     # source line -> file line
statements = {n.lineno + base for n in ast.walk(func) if isinstance(n, ast.stmt)}
statements -= {func.lineno + base}                          # ignore the def line
ifs = [n for n in ast.walk(func) if isinstance(n, ast.If)]
vg = len(ifs) + 1                                           # V(G) = P + 1

def run(args):
    lines = []

    def tracer(frame, event, arg):
        if frame.f_code is certificate_status.__code__:
            if event == "line":
                lines.append(frame.f_lineno)
            return tracer

    sys.settrace(tracer)
    result = certificate_status(*args)
    sys.settrace(None)
    return result, tuple(lines)

def report(tests, rows=True):
    hit, paths = set(), set()
    for name, args, expected in tests:
        result, lines = run(args)
        hit.update(lines)
        paths.add(lines)
        if rows:
            print(f"{name:<5}{str(args):<19}{expected:<20}{result:<20}"
                  f"{'PASS' if result == expected else 'FAIL'}")
    branches = sum((n.body[0].lineno + base in hit)
                   + (n.orelse[0].lineno + base in hit) for n in ifs)
    for name, done, total in [("Statement", len(statements & hit), len(statements)),
                              ("Branch", branches, 2 * len(ifs)),
                              ("Path", len(paths), vg)]:
        pct = 100 * done / total
        print(f"  {name + ' coverage':<20}{done}/{total} = {pct:.1f}%")

print(f"CYCLOMATIC COMPLEXITY: P = {len(ifs)}, V(G) = P + 1 = {vg}\n")
print("All 4 test cases")
print(f"{'TC':<5}{'Input':<19}{'Expected':<20}{'Actual':<20}Result")
report(TESTS)
print("\nOnly TC1 and TC2")
report(TESTS[:2], rows=False)
```

❖ Output: -

```
CYCLOMATIC COMPLEXITY: P = 3, V(G) = P + 1 = 4

All 4 test cases
TC   Input              Expected            Actual              Result
TC1  (90, 70, True)     Certificate Issued  Certificate Issued  PASS
TC2  (90, 70, False)    Submit Assignments  Submit Assignments  PASS
TC3  (90, 30, True)     Retake Quiz         Retake Quiz         PASS
TC4  (50, 70, True)     Course Incomplete   Course Incomplete   PASS
  Statement coverage  8/8 = 100.0%
  Branch coverage     6/6 = 100.0%
  Path coverage       4/4 = 100.0%

Only TC1 and TC2
  Statement coverage  6/8 = 75.0%
  Branch coverage     4/6 = 66.7%
  Path coverage       2/4 = 50.0%
```

❖ Conclusion: -

V(G) is 4. Four test cases give 100% coverage; TC1 and TC2 alone give only 75%, 66.7% and 50%.

Practical – 10

Aim: - Design and develop test suites for a given software module by applying various testing techniques.

❖ Problem: -

Design a test suite for the **LearnSphere course module** using four testing techniques: Equivalence Partitioning, Boundary Value Analysis, Decision Table Testing and Error Guessing.

- **course_fee(price, coupon_percent):** price must be from 1 to 50000 and coupon from 0 to 50 (percent). It returns the fee after discount.
- **enrollment_status(is_logged_in, payment_done, seats_left):** returns Login Required, Payment Pending, Enrolled or Waitlisted.

❖ Test design: -

Input	Valid class	Invalid classes
price	1 to 50000	below 1, above 50000, not a number
coupon_percent	0 to 50	below 0, above 50, not a number
seats_left	0 or more (whole number)	negative number, not a whole number

Boundary	Values tested
price (min 1, max 50000)	0, 1, 2, 49999, 50000, 50001
coupon_percent (min 0, max 50)	-1, 0, 1, 49, 50, 51

Decision table	R1	R2	R3	R4
Logged in?	No	Yes	Yes	Yes
Payment done?	-	No	Yes	Yes
Seats available?	-	-	Yes	No
Result	Login Required	Payment Pending	Enrolled	Waitlisted

❖ Module under test (course_module.py): -

```
"""Module under test: LearnSphere course fee and enrollment rules."""

def course_fee(price, coupon_percent=0):
    """Fee after discount. price: 1 to 50000, coupon: 0 to 50 (percent)."""
    for value in (price, coupon_percent):
        if isinstance(value, bool) or not isinstance(value, (int, float)):
            raise TypeError("price and coupon must be numbers")
    if not 1 <= price <= 50000:
        raise ValueError("price must be between 1 and 50000")
    if not 0 <= coupon_percent <= 50:
        raise ValueError("coupon must be between 0 and 50")
    return round(price - price * coupon_percent / 100, 2)

def enrollment_status(is_logged_in, payment_done, seats_left):
    """What happens when the student presses the Enroll button."""
    if not isinstance(seats_left, int) or isinstance(seats_left, bool):
        raise TypeError("seats_left must be an integer")
    if seats_left < 0:
        raise ValueError("seats_left cannot be negative")
    if not is_logged_in:
        return "Login Required"
    if not payment_done:
        return "Payment Pending"
    return "Enrolled" if seats_left > 0 else "Waitlisted"
```

❖ Test suite (test_suite.py): -

```
"""Practical 10: test suite for course_module.py using four testing techniques."""
from course_module import course_fee as fee, enrollment_status as status

V, T = "ValueError", "TypeError"

# each case: (description, function, arguments, expected result or error name)
EP = [  # equivalence partitioning: one value from every valid / invalid class
    ("valid price, valid coupon", fee, (2000, 10), 1800.0),
    ("valid price, no coupon", fee, (2000,), 2000.0),
    ("price below range", fee, (-500, 10), V),
    ("price above range", fee, (90000, 10), V),
    ("coupon below range", fee, (2000, -5), V),
    ("coupon above range", fee, (2000, 80), V),
]
BVA = [  # boundary value analysis: price 1 and 50000, coupon 0 and 50
    ("price 0 (below min)", fee, (0, 10), V),
    ("price 1 (min)", fee, (1, 0), 1.0),
    ("price 2 (above min)", fee, (2, 0), 2.0),
    ("price 49999 (below max)", fee, (49999, 0), 49999.0),
    ("price 50000 (max)", fee, (50000, 0), 50000.0),
    ("price 50001 (above max)", fee, (50001, 0), V),
    ("coupon -1 (below min)", fee, (1000, -1), V),
    ("coupon 0 (min)", fee, (1000, 0), 1000.0),
    ("coupon 1 (above min)", fee, (1000, 1), 990.0),
    ("coupon 49 (below max)", fee, (1000, 49), 510.0),
    ("coupon 50 (max)", fee, (1000, 50), 500.0),
    ("coupon 51 (above max)", fee, (1000, 51), V),
]
DT = [  # decision table: logged in / payment done / seats available
    ("R1 not logged in", status, (False, True, 5), "Login Required"),
    ("R1 not logged in, full", status, (False, False, 0), "Login Required"),
    ("R2 payment pending", status, (True, False, 5), "Payment Pending"),
    ("R2 payment pending, full", status, (True, False, 0), "Payment Pending"),
    ("R3 paid, seats available", status, (True, True, 5), "Enrolled"),
    ("R4 paid, no seats", status, (True, True, 0), "Waitlisted"),
]
EG = [  # error guessing: inputs a developer may forget
    ("price as string", fee, ("2000", 10), T),
    ("price is None", fee, (None, 10), T),
    ("price is True", fee, (True, 0), T),
    ("coupon as string", fee, (2000, "10"), T),
    ("decimal price and coupon", fee, (999.99, 33.5), 664.99),
    ("negative seats", status, (True, True, -1), V),
    ("seats as string", status, (True, True, "5"), T),
    ("seats as float", status, (True, True, 2.5), T),
]

def execute(func, args):
    try:
        return func(*args)
    except Exception as error:
        return type(error).__name__

total = passed = 0
summary = []
for title, prefix, cases in [("EQUIVALENCE PARTITIONING", "EP", EP),
                             ("BOUNDARY VALUE ANALYSIS", "BVA", BVA),
                             ("DECISION TABLE TESTING", "DT", DT),
                             ("ERROR GUESSING", "EG", EG)]:
    print(f"\n{title}")
    print(f"{'ID':<8}{'Description':<28}{'Expected':<17}{'Actual':<17}Result")
    ok_count = 0
    for no, (desc, func, args, expected) in enumerate(cases, 1):
        actual = execute(func, args)
        ok_count += actual == expected
        print(f"{prefix}-{no:02d}".ljust(8) + f"{desc:<28}{str(expected):<17}"
              f"{str(actual):<17}{'PASS' if actual == expected else 'FAIL'}")
    summary.append((title, len(cases), ok_count))

print(f"\n{'Technique':<30}{'Total':>6}{'Passed':>8}{'Failed':>8}")
for title, count, ok in summary:
    print(f"{title:<30}{count:>6}{ok:>8}{count - ok:>8}")
total, passed = sum(s[1] for s in summary), sum(s[2] for s in summary)
print(f"{'TOTAL':<30}{total:>6}{passed:>8}{total - passed:>8}")
```

```
print("RESULT:", "ALL TESTS PASSED" if total == passed else "SOME TESTS FAILED")
```

❖ Output: -

```
EQUIVALENCE PARTITIONING
ID      Description                 Expected         Actual           Result
EP-01   valid price, valid coupon   1800.0           1800.0           PASS
EP-02   valid price, no coupon      2000.0           2000.0           PASS
EP-03   price below range           ValueError       ValueError       PASS
EP-04   price above range           ValueError       ValueError       PASS
EP-05   coupon below range          ValueError       ValueError       PASS
EP-06   coupon above range          ValueError       ValueError       PASS

BOUNDARY VALUE ANALYSIS
ID      Description                 Expected         Actual           Result
BVA-01  price 0 (below min)         ValueError       ValueError       PASS
BVA-02  price 1 (min)               1.0              1.0              PASS
BVA-03  price 2 (above min)         2.0              2.0              PASS
BVA-04  price 49999 (below max)     49999.0          49999.0          PASS
BVA-05  price 50000 (max)           50000.0          50000.0          PASS
BVA-06  price 50001 (above max)     ValueError       ValueError       PASS
BVA-07  coupon -1 (below min)       ValueError       ValueError       PASS
BVA-08  coupon 0 (min)              1000.0           1000.0           PASS
BVA-09  coupon 1 (above min)        990.0            990.0            PASS
BVA-10  coupon 49 (below max)       510.0            510.0            PASS
BVA-11  coupon 50 (max)             500.0            500.0            PASS
BVA-12  coupon 51 (above max)       ValueError       ValueError       PASS

DECISION TABLE TESTING
ID      Description                 Expected         Actual           Result
DT-01   R1 not logged in            Login Required   Login Required   PASS
DT-02   R1 not logged in, full      Login Required   Login Required   PASS
DT-03   R2 payment pending          Payment Pending  Payment Pending  PASS
DT-04   R2 payment pending, full    Payment Pending  Payment Pending  PASS
DT-05   R3 paid, seats available    Enrolled         Enrolled         PASS
DT-06   R4 paid, no seats           Waitlisted       Waitlisted       PASS

ERROR GUESSING
ID      Description                 Expected         Actual           Result
EG-01   price as string             TypeError        TypeError        PASS
EG-02   price is None               TypeError        TypeError        PASS
EG-03   price is True               TypeError        TypeError        PASS
EG-04   coupon as string            TypeError        TypeError        PASS
EG-05   decimal price and coupon    664.99           664.99           PASS
EG-06   negative seats              ValueError       ValueError       PASS
EG-07   seats as string             TypeError        TypeError        PASS
EG-08   seats as float              TypeError        TypeError        PASS

Technique                      Total  Passed  Failed
EQUIVALENCE PARTITIONING           6       6       0
BOUNDARY VALUE ANALYSIS           12      12       0
DECISION TABLE TESTING             6       6       0
ERROR GUESSING                     8       8       0
TOTAL                             32      32       0
RESULT: ALL TESTS PASSED
```

❖ Conclusion: -

A test suite of 32 test cases was designed with four techniques and all of them passed.

Practical – 11

Aim: - Test the login, Registration (Sign Up), Sign-in functionality of an OAuth module.

❖ Problem: -

Test the **Registration (Sign Up)**, e-mail OTP verification, **Login (Sign In)**, session handling and the **"Continue with Google" (OAuth 2.0)** login of the LearnSphere authentication module. A simple in-memory module is used with a fake Google provider and a fake clock, so no real accounts are needed and expiry can be tested instantly.

❖ Authentication module under test (learnsphere_auth.py): -

```
"""LearnSphere auth module (in-memory): sign up, OTP, login, session, Google."""
import hashlib
import re
import secrets

class AuthError(Exception):
    pass

class FakeGoogle:
    """Stands in for Google: gives one-time codes that expire in 5 minutes."""

    def __init__(self, clock):
        self.clock, self.codes = clock, {}

    def issue_code(self, email, name):
        code = secrets.token_hex(6)
        self.codes[code] = (email, name, self.clock() + 300)
        return code

    def exchange(self, code):
        if code not in self.codes:
            raise AuthError("invalid code")
        email, name, expiry = self.codes.pop(code)          # one time use only
        if self.clock() >= expiry:
            raise AuthError("code expired")
        return email, name

class Auth:
    def __init__(self, clock, google):
        self.clock, self.google = clock, google
        self.users, self.sessions, self.states, self.mailbox = {}, {}, set(), {}

    def _hash(self, pwd, salt):
        return hashlib.pbkdf2_hmac("sha256", pwd.encode(), salt, 1000).hex()

    def _session(self, email):
        token = secrets.token_hex(16)
        self.sessions[token] = (email, self.clock() + 1800)       # 30 minutes
        return token

    # ---------- sign up and e-mail OTP ----------
    def register(self, name, email, pwd, confirm):
        if not re.fullmatch(r"[A-Za-z ]{2,50}", name.strip()):
            raise AuthError("invalid name")
        if not re.fullmatch(r"[^@\s]+@[^@\s]+\.[A-Za-z]{2,}", email):
            raise AuthError("invalid email")
        email = email.lower()
        if email in self.users:
            raise AuthError("email already registered")
        rules = [len(pwd) >= 8, re.search("[A-Z]", pwd), re.search("[a-z]", pwd),
                 re.search(r"\d", pwd), re.search(r"\W", pwd)]
        if not all(rules):
            raise AuthError("weak password")
        if pwd != confirm:
```

```
            raise AuthError("passwords do not match")
        salt = secrets.token_bytes(8)
        self.users[email] = dict(name=name.strip(), salt=salt, verified=False,
                                 hash=self._hash(pwd, salt), fails=0,
                                 locked_until=0)
        self.send_otp(email)

    def send_otp(self, email):
        otp = str(secrets.randbelow(900000) + 100000)
        expiry = self.clock() + 300
        self.users[email].update(otp=otp, otp_expiry=expiry, otp_tries=0)
        self.mailbox[email] = otp                                    # "sent" by mail

    def verify(self, email, otp):
        user = self.users[email]
        if user["otp_tries"] >= 3:
            raise AuthError("too many attempts")
        if self.clock() >= user["otp_expiry"]:
            raise AuthError("otp expired")
        if otp != user["otp"]:
            user["otp_tries"] += 1
            raise AuthError("wrong otp")
        user["verified"] = True

    # ---------- login, session ----------
    def login(self, email, pwd):
        email = email.lower()
        user = self.users.get(email)
        if user is None:
            raise AuthError("invalid credentials")
        if self.clock() < user["locked_until"]:
            raise AuthError("account locked")
        if self._hash(pwd, user["salt"]) != user["hash"]:
            user["fails"] += 1
            if user["fails"] == 5:                              # lock for 15 minutes
                user["locked_until"], user["fails"] = self.clock() + 900, 0
            raise AuthError("invalid credentials")
        if not user["verified"]:
            raise AuthError("email not verified")
        user["fails"] = 0
        return self._session(email)

    def whoami(self, token):
        if token not in self.sessions:
            raise AuthError("invalid session")
        email, expiry = self.sessions[token]
        if self.clock() >= expiry:
            del self.sessions[token]
            raise AuthError("session expired")
        return self.users[email]["name"]

    def logout(self, token):
        self.sessions.pop(token, None)

    # ---------- Continue with Google (OAuth 2.0) ----------
    def google_start(self):
        state = secrets.token_hex(8)
        self.states.add(state)
        return state

    def google_callback(self, code, state):
        if state not in self.states:
            raise AuthError("invalid state")
        self.states.remove(state)                                  # state works once
        email, name = self.google.exchange(code)
        self.users.setdefault(email, dict(name=name, salt=b"", hash="",
                                          verified=True, fails=0, locked_until=0))
        return self._session(email)
```

❖ Test suite (test_auth.py): -

```
"""Practical 11: tests for learnsphere_auth.py (sign up, login, OAuth)."""
import unittest

from learnsphere_auth import Auth, AuthError, FakeGoogle
```

```
EMAIL, PWD = "riya.shah@gmail.com", "Learn@123"
GMAIL = "kunal.joshi@gmail.com"

class Clock:
    """Fake clock so that expiry can be tested without waiting."""

    def __init__(self):
        self.now = 1000.0

    def __call__(self):
        return self.now

    def move(self, seconds):
        self.now += seconds

class Base(unittest.TestCase):
    def setUp(self):
        self.clock = Clock()
        self.google = FakeGoogle(self.clock)
        self.auth = Auth(self.clock, self.google)

    def fails(self, message, func, *args):
        with self.assertRaises(AuthError) as error:
            func(*args)
        self.assertEqual(str(error.exception), message)

    def signup(self, verified=True):
        self.auth.register("Riya Shah", EMAIL, PWD, PWD)
        if verified:
            self.auth.verify(EMAIL, self.auth.mailbox[EMAIL])

class SignUp(Base):
    def test_01(self):
        """REG-01 Valid details are accepted"""
        self.signup(verified=False)
        self.assertFalse(self.auth.users[EMAIL]["verified"])
    def test_02(self):
        """REG-02 Duplicate e-mail is rejected"""
        self.signup(verified=False)
        self.fails("email already registered", self.auth.register,
                   "A B", EMAIL, PWD, PWD)
    def test_03(self):
        """REG-03 Duplicate e-mail in capital letters is rejected"""
        self.signup(verified=False)
        self.fails("email already registered", self.auth.register,
                   "A B", EMAIL.upper(), PWD, PWD)
    def test_04(self):
        """REG-04 E-mail without @ is rejected"""
        self.fails("invalid email", self.auth.register,
                   "Riya Shah", "riya.gmail.com", PWD, PWD)
    def test_05(self):
        """REG-05 Name with digits is rejected"""
        self.fails("invalid name", self.auth.register, "Riya123", EMAIL, PWD, PWD)
    def test_06(self):
        """REG-06 Password of 7 characters is rejected"""
        self.fails("weak password", self.auth.register,
                   "Riya Shah", EMAIL, "Lea@123", "Lea@123")
    def test_07(self):
        """REG-07 Password of exactly 8 characters is accepted"""
        self.auth.register("Riya Shah", EMAIL, "Learn@12", "Learn@12")
    def test_08(self):
        """REG-08 Password without special character is rejected"""
        self.fails("weak password", self.auth.register,
                   "Riya Shah", EMAIL, "Learn1234", "Learn1234")
    def test_09(self):
        """REG-09 Confirm password not matching is rejected"""
        self.fails("passwords do not match", self.auth.register,
                   "Riya Shah", EMAIL, PWD, "Learn@124")
    def test_10(self):
        """REG-10 Password is stored as hash, not plain text"""
        self.signup(verified=False)
        self.assertNotIn(PWD, str(self.auth.users[EMAIL]))

class OTP(Base):
    def setUp(self):
        super().setUp()
```

```
        self.signup(verified=False)
        self.otp = self.auth.mailbox[EMAIL]
    def test_01(self):
        """OTP-01 Correct OTP verifies the account"""
        self.auth.verify(EMAIL, self.otp)
        self.assertTrue(self.auth.users[EMAIL]["verified"])
    def test_02(self):
        """OTP-02 Wrong OTP is rejected"""
        self.fails("wrong otp", self.auth.verify, EMAIL, "000000")
    def test_03(self):
        """OTP-03 OTP still works at 299 seconds"""
        self.clock.move(299)
        self.auth.verify(EMAIL, self.otp)
    def test_04(self):
        """OTP-04 OTP expires at 300 seconds"""
        self.clock.move(300)
        self.fails("otp expired", self.auth.verify, EMAIL, self.otp)
    def test_05(self):
        """OTP-05 Blocked after 3 wrong attempts"""
        for _ in range(3):
            self.fails("wrong otp", self.auth.verify, EMAIL, "000000")
        self.fails("too many attempts", self.auth.verify, EMAIL, self.otp)
    def test_06(self):
        """OTP-06 Resent OTP works after the old one expired"""
        self.clock.move(400)
        self.auth.send_otp(EMAIL)
        self.auth.verify(EMAIL, self.auth.mailbox[EMAIL])

class Login(Base):
    def setUp(self):
        super().setUp()
        self.signup()
    def test_01(self):
        """LOG-01 Login with correct e-mail and password"""
        self.assertEqual(self.auth.whoami(self.auth.login(EMAIL, PWD)), "Riya Shah")
    def test_02(self):
        """LOG-02 Wrong password is rejected"""
        self.fails("invalid credentials", self.auth.login, EMAIL, "Wrong@123")
    def test_03(self):
        """LOG-03 Unknown e-mail gives the same error message"""
        self.fails("invalid credentials", self.auth.login, "nobody@gmail.com", PWD)
    def test_04(self):
        """LOG-04 E-mail is not case sensitive"""
        self.assertTrue(self.auth.login(EMAIL.upper(), PWD))
    def test_05(self):
        """LOG-05 Login without e-mail verification is rejected"""
        self.auth.register("Mehul Patel", "mehul@gmail.com", PWD, PWD)
        self.fails("email not verified", self.auth.login, "mehul@gmail.com", PWD)
    def test_06(self):
        """LOG-06 Account locks after 5 wrong passwords"""
        for _ in range(5):
            self.fails("invalid credentials", self.auth.login, EMAIL, "Wrong@123")
        self.fails("account locked", self.auth.login, EMAIL, PWD)
    def test_07(self):
        """LOG-07 4 wrong passwords then correct one works"""
        for _ in range(4):
            self.fails("invalid credentials", self.auth.login, EMAIL, "Wrong@123")
        self.assertTrue(self.auth.login(EMAIL, PWD))
    def test_08(self):
        """LOG-08 Locked account opens again after 15 minutes"""
        for _ in range(5):
            self.fails("invalid credentials", self.auth.login, EMAIL, "Wrong@123")
        self.clock.move(900)
        self.assertTrue(self.auth.login(EMAIL, PWD))

class Session(Base):
    def setUp(self):
        super().setUp()
        self.signup()
        self.token = self.auth.login(EMAIL, PWD)
    def test_01(self):
        """SES-01 Fake session token is rejected"""
        self.fails("invalid session", self.auth.whoami, "abc123")
    def test_02(self):
        """SES-02 Session is valid at 1799 seconds"""
        self.clock.move(1799)
```

```
        self.assertEqual(self.auth.whoami(self.token), "Riya Shah")
    def test_03(self):
        """SES-03 Session expires at 1800 seconds"""
        self.clock.move(1800)
        self.fails("session expired", self.auth.whoami, self.token)
    def test_04(self):
        """SES-04 Token cannot be used after logout"""
        self.auth.logout(self.token)
        self.fails("invalid session", self.auth.whoami, self.token)

class GoogleLogin(Base):
    def code(self):
        return self.google.issue_code(GMAIL, "Kunal Joshi")
    def test_01(self):
        """OA-01 Start gives a new state value"""
        self.assertIn(self.auth.google_start(), self.auth.states)
    def test_02(self):
        """OA-02 First Google login creates a verified account"""
        token = self.auth.google_callback(self.code(), self.auth.google_start())
        self.assertEqual(self.auth.whoami(token), "Kunal Joshi")
        self.assertTrue(self.auth.users[GMAIL]["verified"])
    def test_03(self):
        """OA-03 Existing user is not duplicated"""
        self.auth.register("Kunal Joshi", GMAIL, PWD, PWD)
        self.auth.google_callback(self.code(), self.auth.google_start())
        self.assertEqual(len(self.auth.users), 1)
    def test_04(self):
        """OA-04 Wrong state is rejected (CSRF check)"""
        self.auth.google_start()
        self.fails("invalid state", self.auth.google_callback,
                   self.code(), "fake-state")
    def test_05(self):
        """OA-05 Same state cannot be used twice"""
        state = self.auth.google_start()
        self.auth.google_callback(self.code(), state)
        self.fails("invalid state", self.auth.google_callback, self.code(), state)
    def test_06(self):
        """OA-06 Authorization code works only once"""
        code = self.code()
        self.auth.google_callback(code, self.auth.google_start())
        self.fails("invalid code", self.auth.google_callback,
                   code, self.auth.google_start())
    def test_07(self):
        """OA-07 Authorization code expires after 5 minutes"""
        state, code = self.auth.google_start(), self.code()
        self.clock.move(300)
        self.fails("code expired", self.auth.google_callback, code, state)

if __name__ == "__main__":
    groups = [("SIGN UP", SignUp), ("E-MAIL OTP", OTP), ("LOGIN", Login),
              ("SESSION", Session), ("GOOGLE OAUTH", GoogleLogin)]
    print("LEARNSPHERE AUTH MODULE - TEST REPORT")
    rows = []
    for title, cls in groups:
        print(f"\n{title}")
        passed = total = 0
        for test in unittest.defaultTestLoader.loadTestsFromTestCase(cls):
            result = unittest.TestResult()
            test.run(result)
            ok = "PASS" if result.wasSuccessful() else "FAIL"
            print(f"  {test.shortDescription():<62}{ok}")
            total += 1
            passed += result.wasSuccessful()
        rows.append((title, total, passed))
    print(f"\n{'Feature':<26}{'Tests':>6}{'Passed':>8}{'Failed':>8}")
    for title, total, passed in rows:
        print(f"{title:<26}{total:>6}{passed:>8}{total - passed:>8}")
    all_total, all_passed = sum(r[1] for r in rows), sum(r[2] for r in rows)
    print(f"{'TOTAL':<26}{all_total:>6}{all_passed:>8}{all_total - all_passed:>8}")
    print("RESULT:", "ALL TESTS PASSED" if all_total == all_passed else "FAILED")
```

❖ Output: -

```
LEARNSPHERE AUTH MODULE - TEST REPORT

SIGN UP
```

```
  REG-01 Valid details are accepted                          PASS
  REG-02 Duplicate e-mail is rejected                        PASS
  REG-03 Duplicate e-mail in capital letters is rejected     PASS
  REG-04 E-mail without @ is rejected                        PASS
  REG-05 Name with digits is rejected                        PASS
  REG-06 Password of 7 characters is rejected                PASS
  REG-07 Password of exactly 8 characters is accepted        PASS
  REG-08 Password without special character is rejected      PASS
  REG-09 Confirm password not matching is rejected           PASS
  REG-10 Password is stored as hash, not plain text          PASS

E-MAIL OTP
  OTP-01 Correct OTP verifies the account                    PASS
  OTP-02 Wrong OTP is rejected                               PASS
  OTP-03 OTP still works at 299 seconds                      PASS
  OTP-04 OTP expires at 300 seconds                          PASS
  OTP-05 Blocked after 3 wrong attempts                      PASS
  OTP-06 Resent OTP works after the old one expired          PASS

LOGIN
  LOG-01 Login with correct e-mail and password              PASS
  LOG-02 Wrong password is rejected                          PASS
  LOG-03 Unknown e-mail gives the same error message         PASS
  LOG-04 E-mail is not case sensitive                        PASS
  LOG-05 Login without e-mail verification is rejected       PASS
  LOG-06 Account locks after 5 wrong passwords               PASS
  LOG-07 4 wrong passwords then correct one works            PASS
  LOG-08 Locked account opens again after 15 minutes         PASS

SESSION
  SES-01 Fake session token is rejected                      PASS
  SES-02 Session is valid at 1799 seconds                    PASS
  SES-03 Session expires at 1800 seconds                     PASS
  SES-04 Token cannot be used after logout                   PASS

GOOGLE OAUTH
  OA-01 Start gives a new state value                        PASS
  OA-02 First Google login creates a verified account        PASS
  OA-03 Existing user is not duplicated                      PASS
  OA-04 Wrong state is rejected (CSRF check)                 PASS
  OA-05 Same state cannot be used twice                      PASS
  OA-06 Authorization code works only once                   PASS
  OA-07 Authorization code expires after 5 minutes           PASS

Feature                    Tests  Passed  Failed
SIGN UP                       10      10       0
E-MAIL OTP                     6       6       0
LOGIN                          8       8       0
SESSION                        4       4       0
GOOGLE OAUTH                   7       7       0
TOTAL                         35      35       0
RESULT: ALL TESTS PASSED
```

❖ Conclusion: -

Registration, OTP verification, login, session handling and Google OAuth login were tested with 35 test cases and all of them passed.

Practical – 12

Aim: - Use the Blisk tool to test the layout and responsiveness of your website or personal project.

❖ Introduction: -

Blisk is a Chromium based browser made for web developers. It shows a website on many devices at the same time, so the layout and responsiveness can be checked quickly. In this practical the home page of the LearnSphere website was tested.

❖ Website under test: -

- **Website:** LearnSphere home page (index.html) with navigation bar, offer strip, hero section, categories, featured courses, plans table and sign in form.
- **Devices:** Samsung Galaxy S8 (360 x 740), iPhone 12 Pro (390 x 844), iPad (768 x 1024) and Laptop (1366 x 768).

❖ Steps performed: -

- The LearnSphere home page was opened in Blisk and the four devices were selected, so the page was shown on all of them at the same time.
- On every device the navigation bar, hero section, category cards, course cards, plans table and sign in form were checked.
- The page was scrolled sideways to check that there is no horizontal scroll.
- The menu icon and the sign in form were tested with different inputs.
- Screenshots were taken and the problems were noted for the bug report (Practical 13).

❖ Screenshots: -

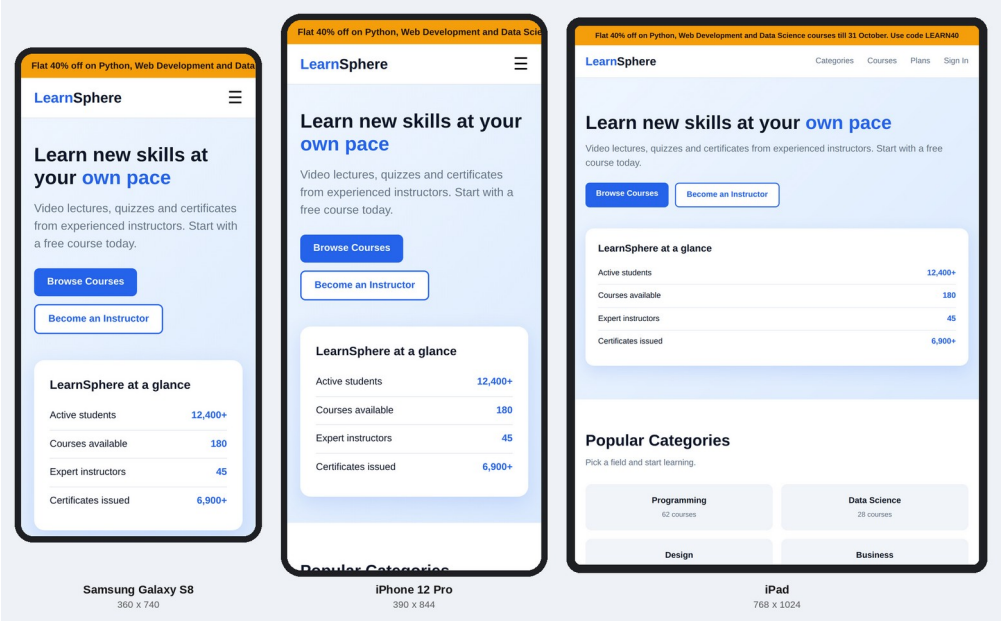


Fig 1: Top of the home page on Galaxy S8, iPhone 12 Pro and iPad

❖ Laptop view and course section: -

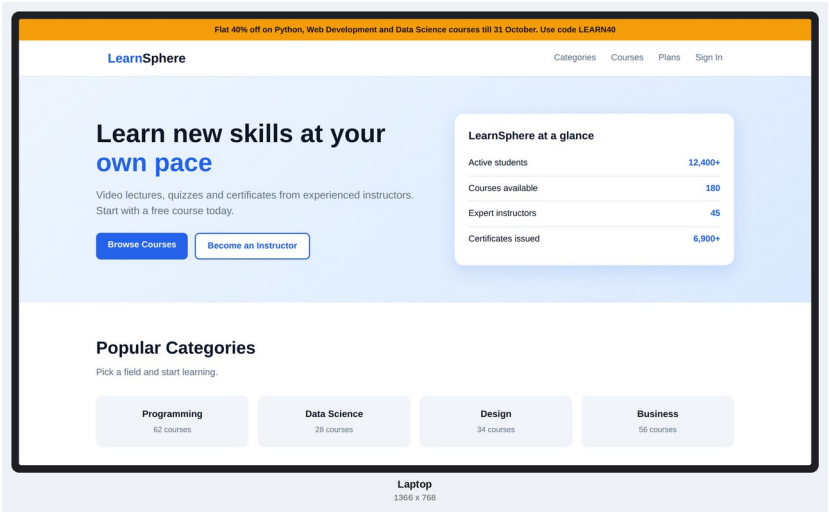


Fig 2: Top of the home page on Laptop

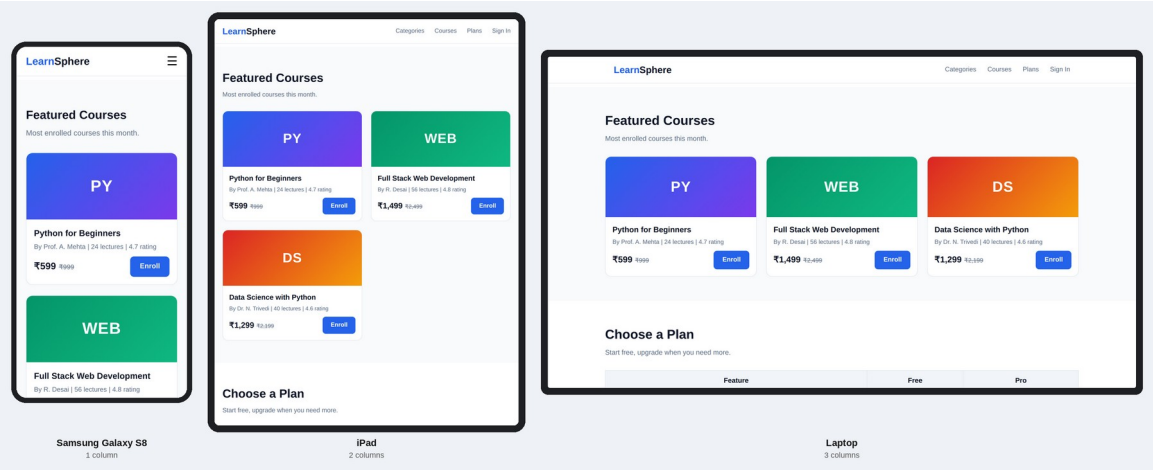


Fig 3: Featured courses section (1, 2 and 3 columns)

❖ Layout and responsiveness results: -

Check	Galaxy S8 (360 px)	iPhone 12 Pro (390 px)	iPad (768 px)	Laptop (1366 px)
Navigation menu	Hamburger icon	Hamburger icon	Full menu	Full menu
Hero section	1 column	1 column	1 column	2 columns
Heading size	30px	30px	36px	44px
Category cards per row	2	2	2	4
Course cards per row	1	1	2	3
Plans table	Scrolls inside its box	Scrolls inside its box	Fits	Fits
Page scrolls sideways	Yes (707 px)	Yes (707 px)	No	No
Result	Issue found	Issue found	Pass	Pass

❖ Breakpoints used in the website (CSS): -

Screen width	Layout
More than 900 px	Full menu, hero in 2 columns, 4 category cards and 3 course cards per row
601 px to 900 px	Full menu, hero in 1 column, 2 category cards and 2 course cards per row
600 px or less	Hamburger icon, 1 course card per row, plans table scrolls inside its box

❖ Functional check of the page: -

Test	Expected result	Actual result	Status
Tap the menu icon on Galaxy S8	Menu opens	Nothing happens, menu stays hidden	Fail
Sign in with e-mail “abc” (no @)	Error for e-mail	Please enter a valid email address.	Pass
Sign in with e-mail “a@b”	Error for e-mail	Login successful. Welcome back!	Fail
Valid e-mail, empty password	Error for password	Login successful. Welcome back!	Fail

❖ Observations: -

- The layout changes properly for tablet and mobile: the menu changes to a hamburger icon and the cards come in 3, 2 and 1 columns.
- The problems found are listed below and reported in detail in Practical 13.

Problem found	Where	Bug ID
Offer strip is wider than the screen, page scrolls sideways	Galaxy S8, iPhone 12 Pro	BUG-001
Menu icon does not open the menu	Galaxy S8, iPhone 12 Pro	BUG-002
Sign in accepts an empty password	All devices	BUG-003
Sign in accepts wrong e-mails like a@b	All devices	BUG-004
Enroll buttons do not open any page	All devices	BUG-005

❖ Conclusion: -

The layout and responsiveness of the LearnSphere website were tested with Blisk on four devices. The page works properly on iPad and Laptop, and problems were found on mobile and in the sign in form. They are reported in Practical 13.

Practical – 13

Aim: - Generate a bug report of any given definition.

❖ Introduction: -

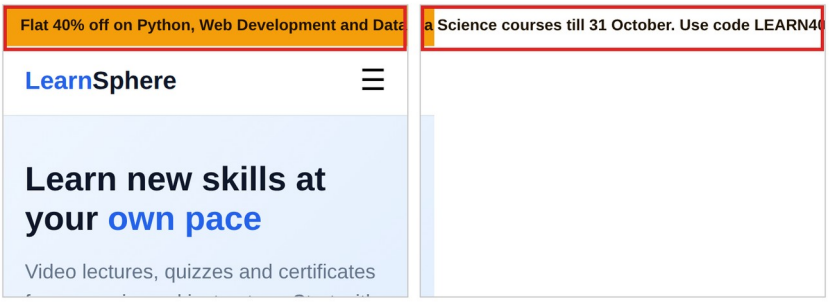
A **bug report** describes a defect found during testing so that the developer can understand it, reproduce it and fix it. The bugs found in the LearnSphere website during Practical 12 are reported below. Severity tells how serious the bug is, and priority tells how soon it should be fixed.

Project: LearnSphere (demo website) **Reported by:** Chavda Divyaraj S. **Date:** 05-10-2026

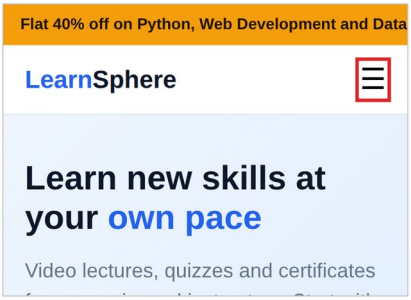
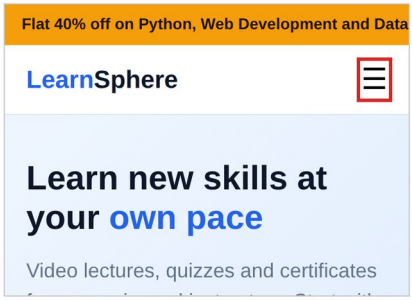
❖ Bug Summary: -

Bug ID	Title	Severity	Priority	Status
BUG-001	Offer strip is wider than the screen on mobile	Medium	Medium	Open
BUG-002	Menu icon does not open the menu on mobile	High	High	Open
BUG-003	Sign in accepts an empty password	High	High	Open
BUG-004	Sign in accepts wrong e-mails like a@b	Medium	Medium	Open
BUG-005	Enroll buttons do not open any page	Low	Medium	Open

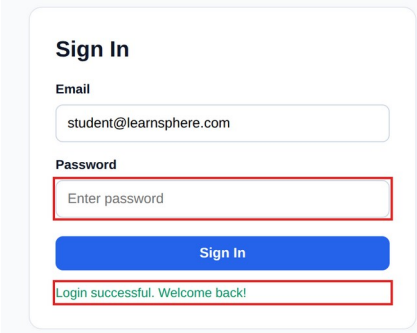
❖ Bug Report 1: -

Bug ID	BUG-001	**Status**	Open
Title	Offer strip is wider than the screen on mobile		
Module	Home page / Offer strip	**Severity**	Medium
Environment	Blisk, Galaxy S8 (360 px) and iPhone 12 Pro (390 px)	**Priority**	Medium
Steps to reproduce	1. Open the home page and select the Galaxy S8 device. 2. Look at the orange offer strip at the top. 3. Scroll the page to the right.		
Expected result	The offer text comes in two or three lines and stays inside the screen. The page does not scroll sideways.		
Actual result	The offer text is on one line and is cut at the right side. The page is 707 px wide on a 360 px screen, so the whole page moves sideways.		
Suggested fix	The offer strip has white-space: nowrap in its CSS. Change it to white-space: normal.		
Screenshot	 Offer text is cut at the right edge After scrolling right: page is wider than the screen		

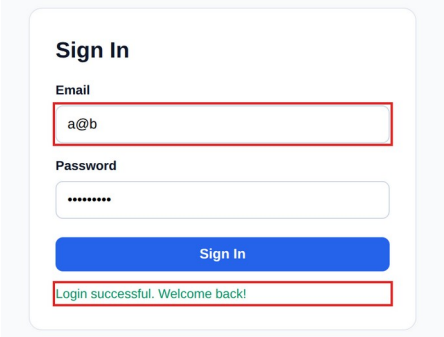
❖ Bug Report 2: -

Bug ID	BUG-002	**Status**	Open
Title	Menu icon does not open the menu on mobile		
Module	Navigation bar	**Severity**	High
Environment	Blisk, Galaxy S8 (360 px)	**Priority**	High
Steps to reproduce	1. Open the home page and select the Galaxy S8 device. 2. Tap the menu (hamburger) icon at the top right.		
Expected result	The menu with Categories, Courses, Plans and Sign In opens.		
Actual result	Nothing happens. The menu stays hidden, so a mobile user can not move to other sections from the menu.		
Suggested fix	The page has no JavaScript click handler for the menu icon. Add a handler that shows and hides the menu.		
Screenshot	 Before tapping the menu icon  After tapping: menu does not open		

❖ Bug Report 3: -

Bug ID	BUG-003	**Status**	Open
Title	Sign in accepts an empty password		
Module	Sign In form	**Severity**	High
Environment	Blisk, Laptop (1366 px)	**Priority**	High
Steps to reproduce	1. Go to the Sign In section. 2. Enter the e-mail student@learnsphere.com. 3. Keep the password empty and click Sign In.		
Expected result	Message like “Password is required” is shown and the user is not logged in.		
Actual result	The message “Login successful. Welcome back!” is shown.		
Suggested fix	Only the e-mail is checked in the script. Check the password also (not empty, minimum 8 characters) and verify it on the server.		
Screenshot			

❖ Bug Report 4: -

Bug ID	BUG-004	**Status**	Open
Title	Sign in accepts wrong e-mails like a@b		
Module	Sign In form	**Severity**	Medium
Environment	Blisk, Laptop (1366 px)	**Priority**	Medium
Steps to reproduce	1. Go to the Sign In section. 2. Enter the e-mail a@b (also abc@ or @) and any password. 3. Click Sign In.		
Expected result	Error message for an invalid e-mail address.		
Actual result	The message “Login successful. Welcome back!” is shown.		
Suggested fix	The script only checks that the e-mail contains @. Use a proper e-mail pattern (name@domain.com).		
Screenshot			

❖ Bug Report 5: -

Bug ID	BUG-005	**Status**	Open
Title	Enroll buttons do not open any page		
Module	Featured courses	**Severity**	Low
Environment	Blisk, Laptop (1366 px)	**Priority**	Medium
Steps to reproduce	1. Scroll to Featured Courses. 2. Click the Enroll button of any course.		
Expected result	The enrollment / payment page of that course opens.		
Actual result	The page only jumps to the top because the button link is “#”.		
Suggested fix	Link every Enroll button to the enrollment page of its course.		

❖ Conclusion: -

Five bugs were found in the LearnSphere website and reported with steps to reproduce, expected and actual result, severity, priority and a suggested fix.